崇实战队 27 赛季算法组第一次作业

第一阶段学习进度报告（待本人核对稿）

学习方向： C 语言基础、C++ 入门、数组与指针、递归、函数设计及基本调试
对应作业： 27 赛季算法组第一次作业（四道编程题）
说明：这份报告依据实际练习、讨论和已知测试记录整理。正式提交前，我会再次核对表述与代码运行结果。

一、学习目标与整体进展

本阶段主要从已有的 C 语言基础过渡到 C++ 编程，通过四道题把数组、指针、循环、函数、递归和输入输出串联起来。此前能理解部分单独语法，但面对完整题目时，容易在函数参数、返回值、运算符和编译环境上出错。此次学习重点不只是写出答案，而是理解代码的执行过程，并通过编译器反馈修正错误。

目前已完成四道编程题的代码，并进行了部分本地运行验证。学习报告及相关提交文件仍需在正式提交前完成最终核对。

二、数组反转：指针与原地修改

本题要求使用指针直接修改原数组，不创建第二个数组。我使用两个指针分别指向数组首尾，通过临时变量交换两端元素，然后让左指针右移、右指针左移，直到两个指针相遇或交错。

主要收获：

- `int* p` 表示指向整数的指针，`p` 保存地址，`*p` 访问该地址上的整数。
- 对 `int*` 指针执行 `p + 1`，表示移动到下一个整数元素，而不是简单地给地址加 1 个字节。
- 数组作为函数实参时，可以通过指针形参修改原数组；`void` 函数也能完成这种修改。
- 必须结合数组长度检查边界，例如 `n=0` 时不能直接计算并解引用末元素指针。

曾遇到的问题：混淆 `arr[0]` 与数组首元素地址；误以为可以像 Python 一样用负下标取得末元素；交换元素时过早移动指针；试图输出 `void` 函数的返回值。通过拆分“取地址—解引用—交换—移动”四个步骤逐一修正。

已在 Windows 上完成编译运行；奇偶长度、空数组等专项测试还值得继续补齐。

三、爬楼梯：递归与迭代

根据每次只能走 1 阶或 2 阶的条件，最后一步可能来自第 `n-1` 阶或第 `n-2` 阶，因此得到：

`f(n) = f(n-1) + f(n-2)`，并约定 `f(0)=1`、`f(1)=1`。

递归实现：在 `n=0` 或 `n=1` 时直接返回 1，其余情况返回两次较小规模函数调用的结果之和。

迭代实现：从已知的两个 1 开始，计算 `next=a+b`，再执行 `a=b`、`b=next`，用循环逐步得到答案。

主要收获：递归需要能够到达的终止条件；函数调用结束后，会回到上一层继续执行。迭代通过循环维护状态，不需要重复展开同一子问题。普通递归实现会产生重复计算，因此输入较大时效率通常不及本题的迭代实现。

曾遇到的问题：把递归和循环混写；在调用函数时重复写 `int`；把 `=` 与 `==` 混淆；忘记声明 `next`；输出比较结果时忽略括号导致运算符优先级错误。

本地运行确认：

n	递归结果	迭代结果	两者一致
0	1	1	true
1	1	1	true
5	8	8	true

输入 10 或 40 时的运行效率对比尚未专门实测。

四、二叉树遍历：递归调用与数组下标

本题不需要定义结构体，而是用数组模拟二叉树。对于下标 `i`，左孩子下标为 `2*i+1`，右孩子下标为 `2*i+2`，`-1` 表示空节点。我选择实现前序遍历和中序遍历。

- **前序遍历：**当前节点 → 左子树 → 右子树。

- **中序遍历：**左子树 → 当前节点 → 右子树。

每次进入递归函数，先检查数组边界与空节点，再根据要求安排当前节点的输出位置及递归调用顺序。理解“左子树处理完成后返回当前节点”是掌握中序遍历的关键。

曾遇到的问题：把 `2*i+2` 错写成 `2*i-1`；把遍历当前节点误理解为再次调用自身并传入相同下标；在中序函数中误调用前序函数；遗漏分号；建树函数误加 `const` 导致无法修改数组。

测试数组 `{1,2,3,4,5,-1,-1}`，本地运行结果为：

```
Preorder: 1 2 4 5 3
Inorder: 4 2 5 1 3
```

结果与预期一致。

五、成绩统计与安全查询：函数分工和边界处理

本题将一个较长的程序拆分为七个函数：初始化成绩、录入成绩、计算平均分、安全查询、分数段统计、查找最高最低分、输出结果。

主要收获：

- 用 `-1` 表示尚未录入成绩，因为 0 分也是有效成绩。
- 查询前先判断 `index<0` 或 `index>=n`，防止越界访问。
- 计算平均分时只统计有效成绩，并用 `static_cast<double>` 避免整数除法截断小数。
- 查找最大最小值时将变量地址交给函数，通过 `*max` 和 `*min` 修改调用者的变量。
- 使用 `if / else if / else` 统计互斥的成绩区间；输出函数只负责展示已有结果，不重复计算。
- 输入错误时可清除 `std::cin` 的失败状态并丢弃错误输入，增强程序健壮性。

曾遇到的问题：在安全查询中使用未定义的 `i` 而非形参 `index`；把比较运算符误写成赋值；误写 `calcAverage(*scores,n)`，混淆指针与元素的值；不理解通过 `return` 返回结果和通过指针修改原变量的区别。

程序实现与验证说明：基础算法函数在练习中逐步完成，完整主程序和输入异常处理经过 AI 辅助整合。本人反馈在 Windows 上运行成功；辅助检查环境另使用 C++17 编译器测试了合法成绩、超出 0–100 范围的成绩、输入字母后重试、零名学生和越界查询。

本程序把 5 个分数段约定为：0–59、60–69、70–79、80–89、90–100。这是程序自行选定的规则，原题没有明确要求具体划分标准。没有有效成绩时，平均分返回 0，最大值和最小值保持-1 作为“无有效成绩”标记。

六、调试工具与方法

本阶段在 Windows 上使用 Sublime Text 编写代码，通过 PowerShell 运行 `g++ -std=c++17` 编译 C++ 源文件。曾因将文件保存为 `.c` 导致无法找到 `<iostream>`，也曾因为 PowerShell 停留在 `C:\Windows\System32` 目录而找不到源文件。后续学会从源文件所在文件夹打开 PowerShell，并根据编译器的报错行号检查语法。

逐渐形成的检查顺序为：确认文件后缀与编译器、检查数组边界、核对函数形参及返回类型、确认赋值和比较运算符、实际运行样例与边界用例。

七、阶段反思与下一步计划

本次作业的最大收获，是开始把零散的语法知识转换成能运行的完整程序。当前对指针和递归的理解比开始时更清晰，但书写细节仍容易出错，尤其是 `=` 与 `==`、变量作用域、递归调用函数名、分号和 `void` 函数的用法。

下一步计划：补齐数组反转和爬楼梯的边界测试；尝试比较递归和迭代的耗时；继续熟悉 GitHub 仓库管理；学习更系统的输入校验和代码注释方法。

本报告为基于真实学习过程整理的待核对稿。正式提交前需由本人检查每项事实、测试记录及个人表述。